Charleston Southern University

G.E.D. Phone

Jeffrey Havens

Coin 499

Dr. Valerie Sessions

April 17, 2015

Table of Contents

Project Purpose

Although 90% of American adults own a cell phone, only 64% own a smartphone[1]. Those feature phone users have voice and SMS services, but lack the myriad of non-communication features smartphones provide. This project implements a way to communicate with a server through SMS in order to receive information not otherwise readily available. A user sends an SMS message containing commands to a predefined phone number. The commands, taking the form of a textual user interface, are evaluated by a Python program running on a web server.

Research

G.E.D. Phone is a server side Python script. Python was chosen for ease of development. I feel comfortable with Python, and find that community created modules (libraries) are widely available for just about any task. Twilio, another technology used, has an officially supported Python library. Flask, a web micro framework is utilized to create a web endpoint to communicate with Twilio. Flask provides easy URL routing, which is the basis for communicating with Twilio, as explained in the software section of this documentation.

Twilio is a IaaS that provides an easy way to develop software that interacts with Telecommunications, primarily Voice

and SMS. Twilio provides its users with a dedicated phone number and an easy to use API for interacting with telecommunications events such as phone calls and text messages. This lets programmers create rich phone based experiences without having to worry about dealing directly with hardware. I chose Twilio over its competitors because it offers a flexible, powerful API considered to be the industry standard for cloud communications systems.

The program itself must be hosted on a server. I chose Heroku to host the program for two reasons. First, it offers free hosting for small projects. I am a poor college student, and would rather take free over pay, and would rather have a third party worry about hosting rather than do it myself. Second, it requires little configuration. Code is pushed to the server via Git, and then the dyno (what the company calls a linux environment that each app runs in) configures itself based upon a requirements file. Because the configuration was ironed out on my local machine, which I used for testing, there was no need to configure the server independently[2].

Hardware

A cell phone with voice and SMS capabilities is required as an endpoint for interacting with G.E.D. phone.

A server is required to run the program in a web accessible manner. Heroku, a cloud hosting service, is used.

Software

This project was written in Python 2.7.3. It is the version of Python I prefer.

Twilio is a cloud based platform for creating programs that interact with telecommunications devices.

Code Explanation

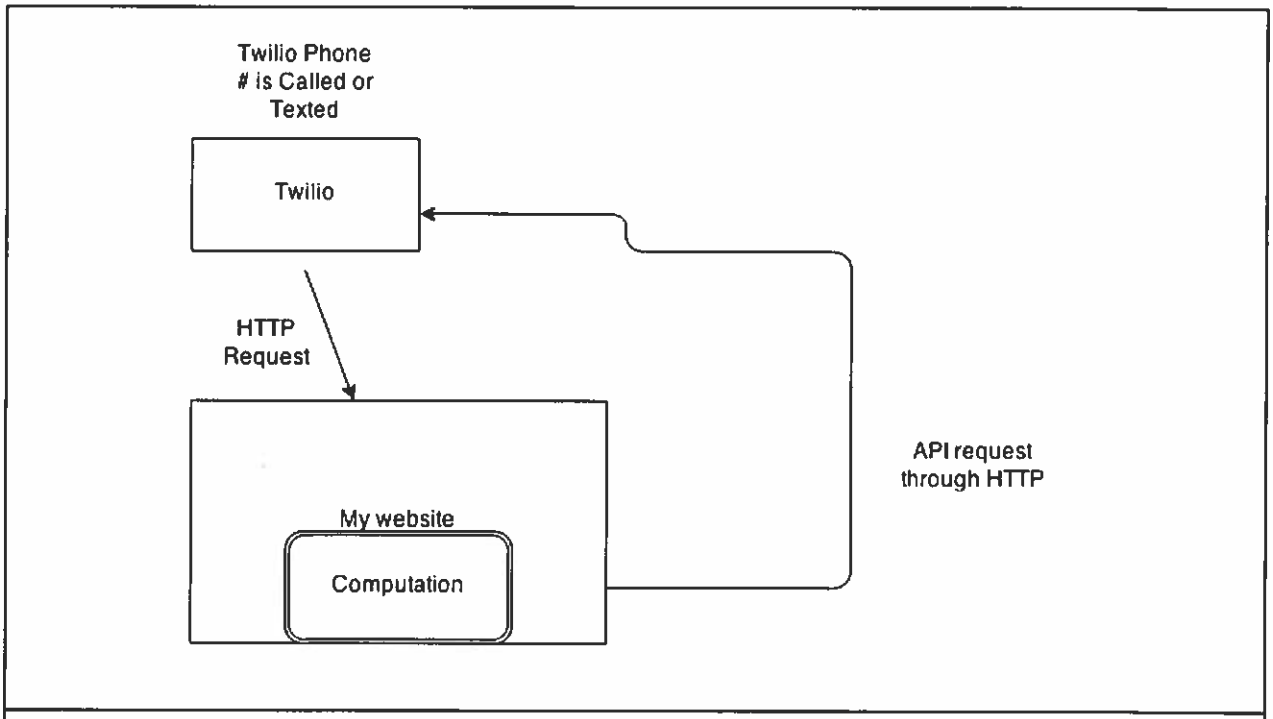

Figure 1: Lifecycle of a G.E.D. phone command

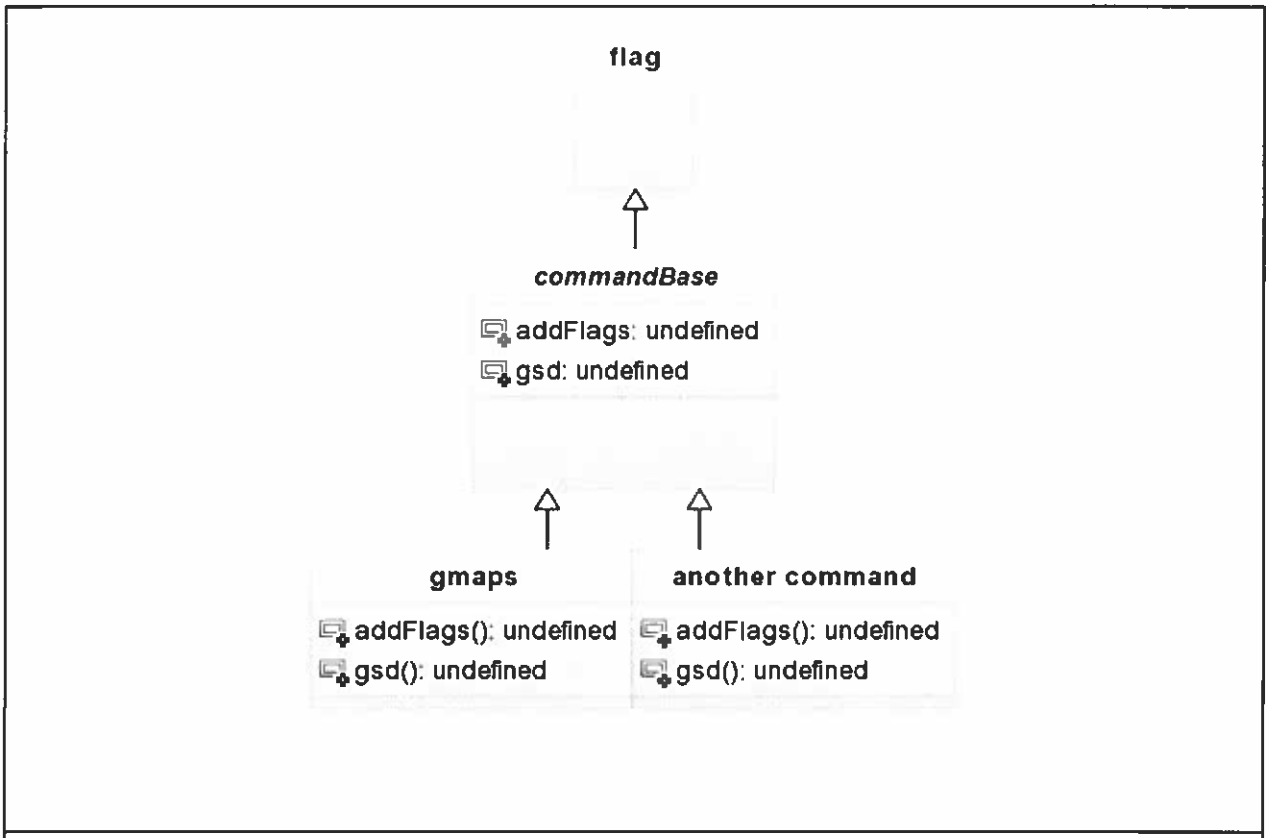

Figure 2: Command inheritance

A G.E.D phone command starts out as a text message to a Twilio number. Twilio forwards this command to the G.E.D. Phone server through an http request (Figure 1). Twilio sends all SMS requests to the /smsendpoint route in app.py

From here the command is split into "flags" by the smsLexer class. A command follows these rules:

- The first character to the first space is a command
- Flags are single words starting with the a single quote

- arguments are whatever characters come directly after a flag or command until another flag or the end
- Example: command an argument 'flag1 'flag2 arg of flag2

Flags are either required or optional. Flags convey information on what to do the same way any command line interface does. Once this is done, the commandRouter class comes into play. It looks to see if any commands match the command specified as the first flag. Also important to note here is the try catch statement. Any errors in the subsequent processing are passed back to the user in a text message.

If the commandRouter has found any suitable matches, it will choose the appropriate command to run. All commands are children of the commandBase class, and therefore must implement the setFlags and gsd methods. This object orientation means that new commands can be written and immediately plugged into the program, making feature development a breeze.

The gsd method in each command does the actual work the command specifies. In the case of the Gmaps command, which gives directions between two addresses, this means getting the directions from the Google directions API. Upon completion, the string containing the response is handed to either the send or

the sendDebug methods. SendDebug is an SMTP server which emails the G.E.D. phone response to a command to my phone number. This was implemented to save money, as each SMS message sent or received charges money to my account. So, using email, which is free and unconnected with Twilio, instead my cell provider, saves money. The response is then split into 160 character chunks, the limit that SMS supports, and sent back to my phone. The command life cycle is then complete.

Challenges Overcome

I learned a great deal during the development of this project. Here are some of the obstacles I had to overcome.

As a text message is limited to 160 characters, the sendDebug method cuts up the response into multiple chunks. The chunks are not guaranteed to be received in the order in which they were sent, so they are numbered. This is an issue because the numbers add characters to the limit of 160, possibly creating the need for an additional chunk. This issue is solved using a guess and check algorithm. The text is split up into 158 character chunks (accounting for the ':' and '/' characters added). Then the numbers are added, and the process is repeated until no new chunks are needed.

I was stuck for a while on a circular import error. Python does not directly tell you there is an error. It instead tells you that it can't find something, even if you explicitly imported it (ImportError: cannot import name foo). This was happening because my import structure was not laid out well. I learned that imports should happen in a top-down manner, not laterally.

I completely re-wrote my lexer. The original lexer was around a hundred lines long and full of convoluted nested if statements. I put it down and picked it up later, when I refactored my code to fix the circular import issue. I completely re-wrote it to a while loop and two if statements. It was a good experience to question my previous reasoning and improve upon it.

References

[1]http://www.pewinternet.org/fact-sheets/mobile-technology-fact-sheet/

[2]http://i.imgur.com/WEz07Js.png

Code

File - C:\Code\warm-hollows-7397 - pretty\app.py

```
import os
import constants
from flask import Flask, request, redirect
import twilio.twiml
import twilio.rest
import inspo
import SendText, twilio.twiml
from smsLexer import smsLexer

app = Flask(__name__)
app.config["PROPAGATE_EXCEPTIONS"] = True


@app.route("/smsendpoint", methods=['GET', 'POST'])
def sms_endpoint():
    constants.baseUrl = "http://desolate-refuge-4358.herokuapp.com/esctwiml"
    lexer = smsLexer()
    #create as a string to avoid unicode headaches
    smsBody = str(request.form["Body"]).lower()
    smsLexer().lexSMS(smsBody)
    #TODO: returnsomething better than a 204
    return ('', 204)

@app.route("/esctwiml", methods=['GET', 'POST'])
def esc_twiml():

    resp = twilio.twiml.Response()
    messageScript = "Jeff, we really need you back at the office. It turns out the dinosaurs were breeding, and nobody can figure out the riddle you left on the system. Please come back with your UNIX knowledge."
    resp.say(messageScript, voice = "woman", language = "en-gb", loop = 10)
    return str(resp)

@app.route("/", methods=['GET', 'POST'])
def hello_world():
    """Respond to incoming SMS with a simple text message."""
    resp = twilio.twiml.Response()
    resp.message("PONG")
    return str(resp)

@app.route("/remind", methods=['GET', 'POST'])
def remind():
    randQuote = inspo.randomQuote()
    SendText.sendDebug(randQuote)
    resp = twilio.twiml.Response()
    resp.message("REMINDER")
    return str(resp)

```

File - C:\Code\warm-hollows-7397 - pretty\app.py

```
47 if __name__ == '__main__':
48     # Bind to PORT if defined, otherwise default to 5000.
49     port = int(os.environ.get('PORT', 5000))
50     app.run(host=constants.host, port=port)
```

```
 1 import urllib
 2 import simplejson
 3 import constants
 4 from commandBase import commandBase
 5 from flag import flag
 6 import time
 7 from twilio.rest import TwilioRestClient
 8
 9
10
11 class esc(commandBase):
12
13     def __init__(self):
14         self.flags = dict()
15         super(esc, self).__init__()
16         self.addFlags()
17
18
19     def addFlags(self):
20         self.flags["esc"] = flag("esc", True)
21         self.flags["txt"] = flag("txt", False)
22         self.flags["sec"] = flag("sec", False)
23
24
25     def assignFlags(self, flagPairs):
26         for possibleFlag in self.flags:
27             if possibleFlag in flagPairs:
28                 flagToFleshOut = self.flags[possibleFlag]
29                 flagToFleshOut.found = True
30                 flagToFleshOut.arg = flagPairs[
   possibleFlag]
31
32
33     def gsd(self, flagPairs):
34         self.addFlags()
35         self.assignFlags(flagPairs)
36
37         if self.flags["esc"].arg is not None:
38             secondsToSleep = int(self.flags["esc"].arg)
39
40             #if the time was given in minutes (default
   behavior)
41             if not self.flags["sec"].found:
42                 secondsToSleep *= 60
43
44             time.sleep(secondsToSleep)
45
46             #return an sms
47             if self.flags['txt'].found:
48                 return self.flags["txt"].arg
49             #make a phone call
50             else:
```

```
51                 client = TwilioRestClient(constants.
   account_sid, constants.auth_token)
52                 sid = client.calls.create(to=constants.
   jMobile,  # Any phone number
53                                 from_=constants.myTwilioNumber
   , # Must be a valid Twilio number
54                                 url=constants.baseUrl + "/
   esctwiml")
55
56                 return
57
58         else:
59             return "esc requires an argument"
```

File - C:\Code\warm-hollows-7397 - pretty\flag.py

```
import lexerException

class  flag:

    def __init__(self, flagName, isRequired):
        self.name = flagName
        self.isRequired = isRequired
        self.arg = None
        self.found = False

    @staticmethod
    def isFlag(word):
        return word[0] is "'"

    def setArg(self, arg):
        if self.hasArg is "yes" and arg is  None:
            raise lexerException("{} must have an arg".
format(self.flagName))
        elif self.hasArg is "no" and arg is not None:
            raise lexerException("{} does not take an arg"
.format(self.flagName))
        else:
            return arg
```

File - C:\Code\warm-hollows-7397 - pretty\gmaps.py

```
import urllib
import simplejson
import constants
from commandBase import commandBase
from flag import flag
import re
import abc


class gmaps(commandBase):

    def __init__(self):
        self.flags = dict()
        super(gmaps, self).__init__()
        self.addFlags()


    def addFlags(self):
        self.flags["gmaps"] = flag("gmaps", False)
        self.flags["f"] = flag("f", True)
        self.flags["t"] = flag("t", True)
        self.flags["i"] = flag("i", False)

    def assignFlags(self, flagPairs):
        for possibleFlag in self.flags:
            if possibleFlag in flagPairs:
                flagToFleshOut = self.flags[possibleFlag]
                flagToFleshOut.found = True
                flagToFleshOut.arg = flagPairs[
possibleFlag]


    def gsd(self, flagPairs):
        self.addFlags()
        self.assignFlags(flagPairs)
        x = 1


        if self.flags["t"].found and self.flags["f"].found
:
            gmapsBaseUrl = "https://maps.googleapis.com/
maps/api/directions/json"
            query_args = {"origin" : self.flags["f"].arg,
"destination" : self.flags["t"].arg, "key": constants.
googleApiKey}
            encoded_args = urllib.urlencode(query_args)
            url = gmapsBaseUrl + "?" + encoded_args
            result = simplejson.load(urllib.urlopen(url))

            legs = result['routes'][0]['legs'][0]
            steps = legs['steps']

```

```
48              stepNumber = 1
49              stepsAppended = ""
50              for step in steps:
51                  stepsAppended += str(stepNumber) + "."
52                  stepsAppended += step['html_instructions']
    + " "
53                  stepNumber += 1
54
55              removeTagRegex =re.compile('<.*?>')
56              stepsAppended = re.sub(removeTagRegex,' ',
    stepsAppended)
57              #remove repeating whitespace
58              stepsAppended = re.sub(' +',' ',stepsAppended)
59
60              if self.flags['i'].found:
61                  distance = " Distance: " + legs['distance'
    ]['text']
62                  duration = " Duration: " + legs['duration'
    ]['text']
63                  stepsAppended += distance
64                  stepsAppended += duration
65              return stepsAppended
66
67          else:
68              return "gmaps requires \'t and \'f"
```

File - C:\Code\warm-hollows-7397 - pretty\Procfile

```
1 web: python app.py
2
```

File - C:\Code\warm-hollows-7397 - pretty\.gitignore

```
1 venv
2 *.pyc
3
```

```
 1 import constants, smtplib, time, math
 2 from email.mime.text import MIMEText
 3 from twilio.rest import TwilioRestClient
 4
 5
 6 def send(text):
 7     client = TwilioRestClient(constants.account_sid,
   constants.auth_token)
 8     sms = client.sms.messages.create(body=text,
 9     to=constants.jMobile,
10     from_=constants.myTwilioNumber)
11
12
13 def sendDebug(text):
14     SMTP_SERVER = constants.gmailServerName
15     SMTP_PORT = constants.gmailServerPort
16     sender = constants.dummyEmailAddr
17     password = constants.dummyEmailPass
18     recipient = constants.jMobileEmail
19     subject = ""
20     headers = ["From: " + sender,
21                "Subject: " + subject,
22                "To: " + recipient,
23                "MIME-Version: 1.0",
24                "Content-Type: text/html"]
25     headers = "\r\n".join(headers)
26     session = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
27     #session.ehlo()
28     session.starttls()
29     session.ehlo
30     session.login(sender, password)
31
32     originalTextLength = len(text)
33     expectedMessageParts = int(math.ceil(len(text) / 160.0
   ))
34     guessMessageParts = 0
35
36     #to insert a notification of a multipart message
37     #e.g. max number of extra spaces for totalMessageParts
    of 10  len("10/10:")
38     #the 2 extra characters are "/" and ":"
39     while guessMessageParts is not expectedMessageParts :
40         guessMessageWorkingLength = originalTextLength
41         charsInOutOf = len(str(expectedMessageParts))
42         for i in range(expectedMessageParts):
43             charsInPartNumber = len(str(i))
44             guessMessageWorkingLength += charsInOutOf +
   charsInPartNumber + 2
45         guessMessageParts = expectedMessageParts
46         expectedMessageParts = int(math.ceil(
   guessMessageWorkingLength / 160.0))
47
```

```
48
49      parts = list()
50      for i in range(expectedMessageParts):
51          messageText = "{}/{}:".format(i+1,
    expectedMessageParts)
52          portionOfFull = 160 - len(messageText)
53          messageText += text[:portionOfFull]
54          parts.append(messageText)
55          text = text[portionOfFull:]
56
57      #does this need ascii instead of utf?
58      for part in parts:
59          print type(part)
60          body = "" + part + ""
61          session.sendmail(sender, recipient, headers + "\r\
    n\r\n" + body)
62      session.quit()
63
64      session.close()
65
```

File - C:\Code\warm-hollows-7397 - pretty\smsLexer.py

```
 1 from flag import flag
 2 from collections import OrderedDict
 3 from lexerException import lexerException
 4 from commandRouter import routeToCommand
 5 class smsLexer(object):
 6
 7     def __init__(self):
 8         return
 9
10     def lexSMS(self, SMS):
11         words = SMS.split()
12         commandArgPairs = OrderedDict()
13
14         commandArgPairs[words[0]] = ""
15         words.remove(words[0])
16
17         while words:
18             if flag.isFlag(words[0]):
19                 #[1:] to remove the leading single quote
20                 commandArgPairs[words[0][1:]] = ""
21             else:
22                 #append the current word to the value
23                 #belonging to the last added key
24                 lastKey = commandArgPairs.keys()[-1]
25                 if commandArgPairs[lastKey] is "":
26                     commandArgPairs[lastKey] += words[0]
27                 else:
28                     commandArgPairs[lastKey] += " " +
   words[0]
29             words.remove(words[0])
30         routeToCommand(commandArgPairs)
31
32
33
34
```

File - C:\Code\warm-hollows-7397 - pretty\constants.py

```
1  ###################################
2  #NOTE: empty strings in this file
3  # containpersonal information
4  #(ex: my email login), which are
5  #be filled out in the "production"
6  #version of the program
7  ###################################
8
9  #local host
10 #host ='127.0.0.1'
11
12 #remote host
13 host='0.0.0.0'
14
15 #Test API Credentials
16 #account_sid = ''
17 #auth_token = ''
18
19 #API Credentials
20 account_sid = ''
21 auth_token = ''
22
23 myTwilioNumber = ''
24 jMobile = ""
25 jMobileEmail = ""
26
27 googleApiKey = ''
28
29 dummyEmailAddr = ""
30 dummyEmailPass = ""
31 gmailServerName = ""
32 gmailServerPort = 587
33
34 #filled in when smsendpoint is hit
35 baseUrl = None
```

File - C:\Code\warm-hollows-7397 - pretty\commandBase.py

```python
import abc

class commandBase:
    __metaclass__ = abc.ABCMeta

    @abc.abstractmethod
    def addFlags(self):
        return

    @abc.abstractmethod
    def gsd(self, flags):
        return

```

File - C:\Code\warm-hollows-7397 - pretty\commandRouter.py

```
 1 from gmaps import gmaps
 2 from esc import esc
 3 import SendText
 4 def routeToCommand(flags):
 5     try:
 6         commands = {'gmaps' : gmaps(), 'esc' : esc()}
 7         command = list(commands.items())[0][0]
 8         command = commands[command]
 9         commandResult = command.gsd(flags)
10         if commandResult is not None:
11             SendText.sendDebug(commandResult)
12     except Exception as e:
13          SendText.sendDebug(str(e))
```

```
1 Flask==0.9
2 Jinja2==2.6
3 Werkzeug==0.8.3
4 wsgiref==0.1.2
5 twilio==3.6.6
6 simplejson==3.3.1
```

File - C:\Code\warm-hollows-7397 - pretty\lexerException.py

```
1 class lexerException(Exception):
2     def __init__(self, value):
3         self.parameter = value
4     def __str__(self):
5         return repr(self.parameter)
```